

MintEval: Do LLMs Implement the Trading Strategy You Asked For?

A Behavioural-Equivalence Benchmark for Natural-Language-to-Strategy Code

[TODO: author list and order must be fixed before the abstract deadline]

[TODO: affiliation]

ABSTRACT

Large language models are moving from producing trading *signals* to writing the code that executes them. The failure mode of the second role is silent: generated code runs, a backtest plots, yet the risk logic that the trader described is not the logic being executed. Existing code benchmarks test functional correctness on unit tests and finance benchmarks test forecasting; neither measures whether an implementation *behaves* like the strategy that was asked for. We introduce MintEval, a benchmark in which reference strategies are generated programmatically from a library of composable building blocks, back-translated into colloquial trader instructions, and re-implemented by the model under test. Generated and reference programs are executed bar by bar on identical market data and frictions, and compared on their actions rather than on code similarity or profit: alpha is differenced away. MintEval v0 contains 800 tasks on BTCUSDT 15-minute data, stratified by an execution-measured state-span complexity τ that is decoupled from description length. Three frontier models from three vendors (Claude Opus 5.5, Qwen3.8-Max, DeepSeek-V4-Pro) reach mean ActionMatch 0.889, 0.819 and 0.782 on a stratified subset of 200 tasks, yet 0.275, 0.390 and 0.405 of their tasks are silent failures: the code runs and trades but behaves differently on more than 10% of active bars. The LLM judge of a recent strategy-generation benchmark, applied verbatim, passes every one of Opus’s silent failures, and passes programs with a single injected error even when that error leaves the strategy almost never trading. Low-cost models degrade with the execution-measured state span τ of a task; frontier models do not, so their residual errors are not a matter of state depth.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation**; • **Computing methodologies** → *Natural language processing*; • **Applied computing** → Forecasting.

KEYWORDS

LLM evaluation, code generation, benchmark, algorithmic trading, behavioural equivalence, stateful programs

1 INTRODUCTION

LLMs are increasingly asked not only for a market view [10, 14, 15] but for the code that turns a trader’s intent into orders. In that role an error is not a bad forecast but an implementation shortfall [11] between the strategy that was intended and the one that runs. Such errors are *silent*: the code compiles, the backtest completes and produces a plausible equity curve, and nothing signals that, say, the breakeven stop never moves or the cooldown resets every bar.

Operational failures of trading code are costly [13], and backtests are already a fragile basis for decisions [2, 3].

Code benchmarks such as HumanEval [4], MBPP [1], EvalPlus [9], DS-1000 [8], BigCodeBench [17] and SWE-bench [6] score functional correctness against unit tests that a human wrote for a precise specification. Trading strategies differ in three ways that make that template insufficient: (i) they are *stateful* over thousands of steps, and the hard part is the persistent state (an entry price, a ratcheted stop, a cooldown clock); (ii) they have *zero tolerance*: a single mis-timed exit changes every later decision; (iii) the specification arrives as *colloquial, jargon-laden* language, not as a docstring.

Contributions. (1) A task definition, $\Phi : q_s \mapsto \hat{c}_s$, scored by bar-level *behavioural equivalence* with a reference program under identical data and frictions. (2) A metric suite led by ActionMatch on active bars. (3) A generator of contamination-free tasks with two separated complexity axes: description length K (generation side) and state span τ and register count R (measured by instrumenting execution). (4) Results for four models in an open setting (instruction + interface only) and a closed diagnostic setting (with a menu of building blocks), separating understanding from implementation.

2 TASK

A task s consists of a reference program c_s , a colloquial instruction q_s and fixed market data D with frictions F . The model under test receives q_s and an interface document (engine semantics, the indicator library, and desk conventions such as “entry price is the close of the signal bar”) and returns $\hat{c}_s$. Both programs implement `strategy(hist, state, pos, ind)`, are called at every bar close, and return a target position in $[-1, 1]$ plus optional stop and take-profit levels for the next bar. We call $\hat{c}_s$ *behaviourally equivalent* to c_s on (D, F) if their quantised target sequences coincide on every bar. Equivalence is a property of behaviour, not of code: two programs with different structure are equivalent if they act identically, and two programs that differ by one comparison operator are not.

The engine enforces causality physically: bar t is copied into the visible buffers right before the call, indicators are revealed bar by bar, and any access to $t+1$ raises an error that cannot be swallowed. State is a flat dictionary of scalars whose reads and writes are logged. Generated code runs in a subprocess sandbox (imports limited to math/numpy, no I/O, 10 s budget).

3 METRIC SUITE

ActionMatch (primary) is the fraction of *active* bars on which the two quantised targets are equal, where a bar is active if either program holds or targets a non-zero position. Restricting to active bars matters: flat periods are trivially matched and would otherwise inflate every score. **TradeF1** matches round trips exactly (direction, entry bar, exit bar). **[ES]** is the absolute difference in total return in basis points of initial equity; we take the absolute value because an implementation error that happens to make money is still an error, and signed errors would cancel across tasks. We also report ΔSharpe and ΔMaxDD . A run that raises, times out or looks ahead is **CompileOK**= 0; its other metrics are left missing rather than imputed. In the closed setting, **SpecMatch** scores the model’s structured reading of q_s slot by slot. A *silent failure* is a run with **CompileOK**= 1 and **ActionMatch**< 0.9. The threshold is a convention, not a cliff: at 0.8 the silent share ranges from 0.370 to 0.741 across models and settings, and at 0.99 from 0.372 to 0.884 (GPT-5.4-mini open: 0.655 at 0.8, 0.729 at 0.9, 0.840 at 0.99).

4 BENCHMARK CONSTRUCTION

Building blocks. The library has 6 signal, 2 filter, 3 sizing and 9 risk blocks, each with a discrete parameter grid and a precise English template. Programs combine one signal, a direction, a filter, a sizing rule and 1–3 risk layers; hard combinations include moving boxes, scaling out, pyramiding, multi-timeframe filters, breakout-then-retest state machines and risk layers that read each other’s state (a trailing stop that arms only after breakeven).

Two complexity axes. K (bits) is the description length of the program in the generator’s grammar. $\tau_{\max}$ and τ_{p90} are the maximum and 90th percentile, over all value reads of the state dictionary during the reference backtest, of the number of bars since the key was last written; R counts keys with at least one cross-bar read. Across the task set $\text{corr}(\log \tau_{\max}, K) = -0.03$. Several blocks vary τ through a parameter alone (time stop, cooldown, retest window), which lets us estimate the effect of τ within a block via block fixed effects.

Selection. We sample 15000 candidate programs and keep the 14393 that make at least 20 round trips, are not flat throughout and do not lose half their equity in the first 5% of bars (607 removed for too few trades, 63 for never trading). Because programs with short state spans trade more often and therefore pay more fees, τ and reference return are confounded in the raw pool. We therefore stratify jointly: within each $\tau_{\max}$ quintile we draw 32 tasks from each of five equal-width bands of reference return, giving 800 tasks with median reference return -0.37, -0.37, -0.37, -0.34 and -0.34 across the five quintiles.

Why losing references do not undermine the measurement.

The median reference program loses -35.1% over the two years and only 8.9% are profitable after costs. This is friction, not a negative edge: re-running the same programs with zero fees and slippage gives a median of -0.6% with 47.8% profitable, i.e. randomly assembled rules with no edge that trade frequently at 6 bp per fill. The measurement does not depend on the sign of P&L: **ActionMatch** and **TradeF1** compare actions, and **[ES]** compares two programs under identical frictions, so a faithful re-implementation of a losing

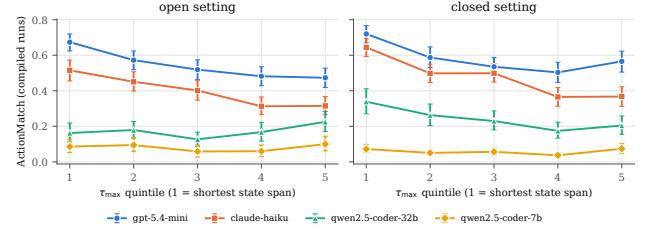


Figure 1: ActionMatch by $\tau_{\max}$ quintile (mean, 95% bootstrap CI).

strategy scores perfectly and an unfaithful one does not. Empirically, **ActionMatch** is only weakly related to reference return, and in the direction opposite to the concern: references that lose more are slightly *easier* to match (-0.11 **ActionMatch** per 100% of return, open setting, $p = 3e - 05$), and the τ coefficient is unchanged when R_{bench} is added as a control (-0.063 open, -0.086 closed, vs. -0.065 and -0.088 without it). A benchmark whose references are realistic trading candidates is nevertheless more convincing for a finance audience; v1 lowers frictions to 2–3 bp and adds 1-hour bars so that the reference return distribution straddles zero.

Back-translation. An LLM translator (a third vendor, not among the models under test) rewrites each program, given its precise template description and source, as a 40–120 word desk message in one of eight rotating voices. Deterministic gates require every parameter value to appear, at least three expressions from a slang dictionary (drafted with LLM help, under human review), no code-like tokens, and the conventional phrasing for formula-level rules; failures are fed back for up to six attempts. A round-trip reader from a fourth vendor then recovers the block-level spec from the message; a mismatch is fed back as well. 796 of 800 messages pass all gates (780 round-trip verified, 20 unverifiable because the reader failed); median length 96 words with 11 slang hits. The reader sees the block menu, so it verifies identification rather than sufficiency; two ambiguities it missed during calibration were resolved with explicit desk conventions (§7).

Contamination. Tasks are sampled from a combinatorial space at build time; neither the instruction nor the reference program exists on the web.

5 PRELIMINARY RESULTS

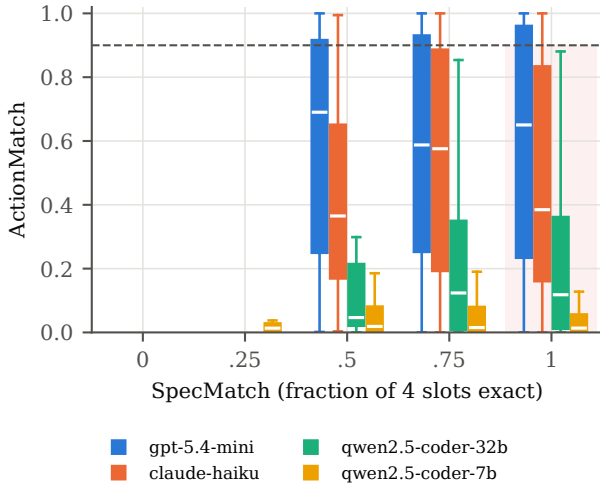
Models and setup. Two low-cost closed models from different vendors (GPT-5.4-mini, Claude Haiku) and two open-weight coders (Qwen2.5-Coder 32B and 7B, served locally with vLLM in batch-invariant mode); temperature 0, one sample per task and setting.

Low-cost models are far from saturation (Table 1). In the open setting the strongest of them reaches **ActionMatch** 0.544 on compiled runs and reproduces 0.087 of tasks exactly; the median **[ES]** is 442 bp of initial equity. Compilation is not the bottleneck for the closed models (**CompileOK** 0.955 and 0.948): most of their failures are silent, 0.729 and 0.806 of all tasks.

Frontier models narrow the gap but do not close it (Table 2, Fig. 3). On a subset of 200 tasks (40 per τ quintile, open setting), Claude Opus 5.5, Qwen3.8-Max and DeepSeek-V4-Pro reach **ActionMatch** 0.889, 0.819 and 0.782 and reproduce 0.575, 0.450 and

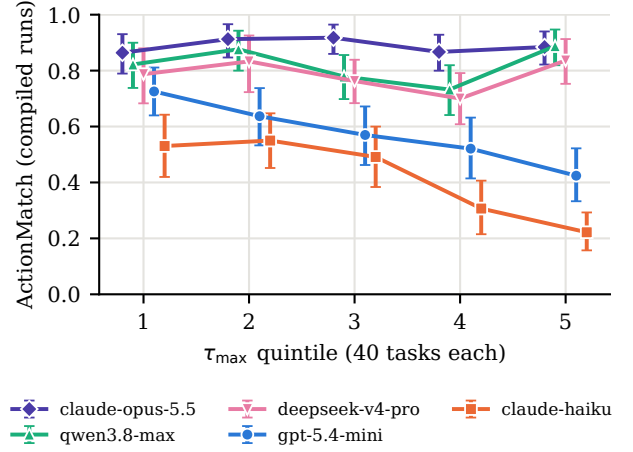
Table 1: Leaderboard on 800 tasks. ActionMatch, TradeF1 and |ES| are over compiled runs; Silent = share of all tasks that compile but have ActionMatch < 0.9.

Model	Set.	CompileOK	SpecMatch	ActionMatch	TradeF1	ES med (bp)	Silent
gpt-5.4-mini	open	0.955	–	0.544	0.469	442	0.729
claude-haiku	open	0.948	–	0.400	0.350	752	0.806
qwen2.5-coder-32b	open	0.601	–	0.172	0.191	1646	0.586
qwen2.5-coder-7b	open	0.374	–	0.081	0.043	2252	0.372
gpt-5.4-mini	closed	0.984	0.938	0.582	0.541	362	0.665
claude-haiku	closed	0.956	0.946	0.476	0.403	613	0.755
qwen2.5-coder-32b	closed	0.613	0.935	0.240	0.185	1360	0.576
qwen2.5-coder-7b	closed	0.488	0.741	0.058	0.021	2781	0.487

**Figure 2: Closed setting: SpecMatch vs ActionMatch per task. Shaded: specification fully correct but behaviour diverges.**

0.330 of tasks exactly, against 0.575 and 0.085 for GPT-5.4-mini on the same tasks; but 0.275, 0.390 and 0.405 of their tasks remain silent failures. None of the three shows a τ gradient: the slope of ActionMatch per decade of $\tau_{\max}$ is +0.017 (95% CI -0.036 to +0.070), +0.010 (-0.052 to +0.072) and +0.018 (-0.053 to +0.088), and each interval excludes the slopes of GPT-5.4-mini (-0.168) and Claude Haiku (-0.194). The flat profile is therefore not noise and not specific to one model family; the τ effect of Section 5 is a property of the low-cost models.

Where frontier silent failures concentrate. For Opus 5.5 they cluster in the stateful breakout-then-retest signal (0.724 silent with it vs. 0.199 without, Fisher $p = 5e - 08$), the higher-timeframe trend filter (0.500 vs. 0.169, $p = 3e - 06$) and the trailing stop (0.434 vs. 0.218, $p = 0.004$), not in the number of stacked risk layers, the moving box (0.156 vs. 0.298) or the cooldown. Part of this concentration is the benchmark’s, not the model’s: in the retest tasks whose instruction does not say that the breakout is measured on the *close*, 12 of 13 Opus runs fail silently (Opus arms on the intrabar high, a defensible reading of “prints a new high”), against 9 of 16 when it does. Our round-trip gate verifies that a reader can *identify*

**Figure 3: ActionMatch by $\tau_{\max}$ quintile on the 200-task subset (mean, 95% bootstrap CI).****Table 2: Same 200 tasks (40 per $\tau_{\max}$ quintile), open setting. ActionMatch and the quintile columns are over compiled runs; Exact and Silent are shares of all tasks.**

Model	CompileOK	ActionMatch	Exact	Silent	τ_1	τ_2	τ_3	τ_4	τ_5
claude-opus-5.5	1.000	0.889	0.575	0.275	0.86	0.91	0.92	0.87	0.88
qwen3.8-max	1.000	0.819	0.450	0.390	0.82	0.88	0.78	0.73	0.89
deepseek-v4-pro	0.910	0.782	0.330	0.405	0.79	0.83	0.76	0.70	0.84
gpt-5.4-mini	0.945	0.575	0.085	0.710	0.73	0.64	0.57	0.52	0.42
claude-haiku	0.955	0.420	0.040	0.820	0.53	0.55	0.49	0.31	0.22
qwen2.5-coder-32b	0.635	0.181	0.005	0.615	0.08	0.22	0.15	0.22	0.22
qwen2.5-coder-7b	0.330	0.076	0.000	0.330	0.07	0.09	0.07	0.04	0.09

each block, not that the message *suffices* to fix its behaviour; at the frontier, that difference matters (Limitations).

Understanding is not the bottleneck (Fig. 2). With the block menu, GPT-5.4-mini, Claude Haiku and Qwen-32B recover the specification with SpecMatch 0.938, 0.946 and 0.935. Yet among the 1767 compiled runs whose specification is fully correct, 0.792 have ActionMatch below 0.9 (GPT-5.4-mini 0.666, Haiku 0.791, Qwen-32B 0.934). The closed setting improves ActionMatch only modestly over the open one (e.g. 0.544 $\rightarrow$ 0.582), so most of the gap is in implementing persistent logic, not in reading the instruction.

State span matters (Fig. 1). For the two stronger models, ActionMatch falls from the shortest to the longest $\tau_{\max}$ quintile (GPT-5.4-mini open $0.674 \rightarrow 0.473$; Haiku open $0.515 \rightarrow 0.315$). Pooling models, an OLS of ActionMatch on $\log_{10} \tau_{\max}$ with model dummies and task-clustered errors gives -0.065 (s.e. 0.013) per decade in the open setting and -0.088 (0.013) in the closed one. Because the generator makes τ nearly orthogonal to description length ($\text{corr} = -0.03$), adding K leaves the estimate unchanged (-0.066 , -0.088). With signal and risk-block fixed effects and register count it shrinks to -0.041 ($p = 0.014$) and -0.058 ($p = 0.00023$); adding McCabe complexity and log trade count leaves -0.033 ($p = 0.049$, borderline) and -0.063 ($p = 3.7e - 05$). The within-block effect is therefore smaller than the raw gradient but does not vanish; the open-setting estimate under full controls is borderline and needs the larger v1 set. The Qwen models are near the floor, where no gradient can show.

What goes wrong. For the open-weight models, failures that raise are dominated by interface misuse (Qwen-7B indexes the history object directly), state-type violations (storing None, which the interface forbids) and look-ahead: indexing a higher-timeframe series with the 15-minute bar index (e.g. `htf.close[t // 4]`), which reads the candle still in progress. In an engine without physical causality enforcement this silently leaks future data into the backtest. Silent failures we inspected include breakout levels computed over a window that contains the current bar (so the breakout can never trigger), stops re-anchored to the current close on every bar instead of to the entry, and HTF filters evaluated on the wrong candle. A systematic taxonomy is left to v1.

A code-reading judge does not see silent failures

We apply the QuantCode-Bench judge [7] verbatim (prompt and rating parser from its public repository; temperature 0, thinking off; the judge model is Claude Sonnet 4, the default in that repository) to every compiled open-setting run of Opus 5.5 and GPT-5.4-mini on the 200-task subset, giving it our instruction q_s and the generated code.

Table 3: QuantCode-Bench judge vs. behavioural equivalence (ActionMatch ≥ 0.9).

Model		Judge pass	Judge fail
Opus 5.5	ActionMatch ≥ 0.9	145	0
	ActionMatch < 0.9	55	0
GPT-5.4-mini	ActionMatch ≥ 0.9	43	0
	ActionMatch < 0.9	120	0

The judge passes every judged run (Table 3): all 55 silent failures of Opus 5.5 (0.275 of its passes) and all 120 of GPT-5.4-mini (0.736 of its passes; 26 of its runs could not be judged because our API budget ran out). The judge is not simply permissive: in a negative control that pairs an instruction with code written for a different task using a different signal family, it rejected 7 of 7. It detects the wrong strategy; it cannot detect the right strategy implemented wrongly. This is the gap behavioural equivalence closes, and it applies to any evaluation whose final check reads code rather than executing it against a reference.

Table 4: Graded negative control: one injected error per program (30 per type). Judge = Kimi K2.5 with the QuantCode-Bench prompt.

Injected error	AM	AM < .9	ES bp	Judge pass
stop at current close	0.577	0.833	472	0.900
window incl. current bar	0.009	1.000	4293	1.000
cooldown reset each bar	0.710	0.667	745	0.900
parameter one step off	0.480	0.800	674	0.900
comparison reversed	0.425	1.000	728	0.900

A second judge and graded errors. Re-running the same prompt with a second judge (Kimi K2.5, thinking off; the OpenAI and Google judges we planned were unavailable to our API account) gives the same picture across all models: it passes 55 of Opus’s 55 silent failures, 77 of 78 for Qwen3.8-Max and 139 of 142 for GPT-5.4-mini; it does reject much of Qwen2.5-Coder-7B’s output (pass rate 0.470). To locate the judge’s resolution we injected one known error into reference programs (Table 4). Every type changes behaviour substantially, yet the judge passes 120 of the 129 behaviourally divergent mutants, including all breakout windows that wrongly contain the current bar, an error that leaves the strategy almost never trading.

What the errors cost. Regressing winsorized $|ES|$ (bp of initial equity) on $\log_{10} \tau_{\max}$, realised volatility over the reference’s holding bars and their interaction, with trade count and K as controls, implementation errors cost more in volatile holding periods for the low-cost models (GPT-5.4-mini open: 161 bp per standard deviation of volatility, $p = 0.01$; Haiku: 375 bp, $p = 2.3e - 07$) and in long-state tasks (561 and 505 bp per decade of τ). The interaction is significant in only 1 of 15 model $\times$ setting cells, which we read as no evidence of amplification in v0; realised volatility varies little within a single two-year window, and the multi-window data of v1 are needed to test it. For Opus 5.5 neither term is distinguishable from zero.

Sensitivity to the state rule. Re-running every program that failed only because it stored None or a string in the state dictionary under a lenient engine raises CompileOK for Qwen-32B from 0.601 to 0.706 (open) and for Qwen-7B from 0.488 to 0.713 (closed), but leaves ActionMatch on compiled runs essentially unchanged (Qwen-32B open 0.172 vs. 0.169): the rescued programs are mostly silent failures. The strict rule moves failures between the loud and the silent column; it does not drive the conclusions.

6 RELATED WORK

Code generation benchmarks score functional correctness on unit tests [1, 4, 8, 9, 17] or on repository tests [6]; LiveCodeBench addresses contamination by collecting problems after model cut-offs [5]. MintEval instead generates tasks combinatorially and judges behaviour over thousands of stateful steps against a reference, which makes silent divergence measurable. **Closest to our work,** QuantCode-Bench [7] collects 400 real strategy descriptions (Reddit, TradingView, StackExchange, GitHub) and checks generated Backtrader code for compilation, backtest execution, the presence of trades, and semantic alignment judged by an LLM; the best models reach 70–76% judge pass in a single turn and 95–98% with iterative repair. Its tasks are real and its model coverage broad, both stronger than

ours. The two benchmarks answer complementary questions: QuantCodeBench asks whether a model can produce an executable strategy that plausibly matches a real description; MintEval asks whether the produced strategy *behaves* as described, which requires a reference program and lets us vary the mechanism. In their pooled single-turn breakdown only 2.7% of attempts are rejected by the judge; a code-reading judge cannot observe a trailing stop that is re-anchored every bar. We measure this gap directly in Section 5. **Code hallucination** studies catalogue incorrect but plausible code [12, 16]; we quantify one consequential form of it. **LLMs in finance** have been studied mainly as forecasters and domain language models [10, 14, 15]. Our metric borrows the logic of implementation shortfall [11]: the gap between the intended and the executed strategy, with alpha differenced away. Backtest overfitting [2, 3] and operational failures [13] motivate treating implementation fidelity as a first-class risk.

7 LIMITATIONS

MintEval is a *diagnostic* benchmark: controlled generation lets us vary τ , K and block type independently, which a benchmark of naturally occurring tasks cannot do, but it does not sample the distribution of real strategies. A real subset (strategies ported from open-source frameworks and exchange product specifications, with human-written intents and human-verified reference ports) is under construction; its role is to test whether model rankings and the τ effect transfer. v0 is small: one instrument (BTCUSD), one data window, 800 tasks and four models with one sample each. A human baseline and a human-rewritten instruction subset are in progress; until then, instructions are LLM-written and may be friendlier to LLMs than real traders’ messages, and the slang dictionary is still under human review. Latency is deliberately excluded: fidelity and latency are orthogonal goals. The unit of observation is the bar-level action and the complete round trip; we do not model intra-position dynamics such as fills inside a bar beyond stop/take priority. The judge comparison uses Claude Sonnet 4 (the default of the original repository) and, as an interim second judge, Kimi K2.5; OpenAI and Google judges were planned but were refused by their providers for our account at the time of writing. DeepSeek-V4-Pro also served as the round-trip reader, which may slightly favour it; Qwen3.8-Max ran at low reasoning effort because its default setting exhausted the token budget. The reference library, interface document and desk conventions were written with the help of Claude Opus 5.5, the same family as our frontier model; shared conventions may favour it, and replicating the frontier result with models from other families is a priority for v1. Finally, a mismatch can stem from an ambiguous instruction rather than from the model; the round-trip reader bounds but does not eliminate this.

ETHICS STATEMENT

MintEval evaluates implementation fidelity, not profitability; no strategy in it is investment advice, and most reference strategies lose money after costs. Market data are public (data.binance.vision). No personal data are used. Model-generated code runs in a sandbox without network or file access.

REFERENCES

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, et al. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).
- [2] David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2014. Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance. *Notices of the American Mathematical Society* 61, 5 (2014), 458. <https://doi.org/10.1090/noti1105>
- [3] David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2016. The Probability of Backtest Overfitting. *The Journal of Computational Finance* (2016). <https://doi.org/10.21314/JCF.2016.322>
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [5] Naman Jain, King Han, Alex Gu, Wen-Ding Li, et al. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv preprint arXiv:2403.07974* (2024).
- [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.06770.
- [7] Alexey Khoroshilov, Alexey Chernysh, Orkhan Ekhtibarov, Nini Kamkia, and Dmitry Zmitrovich. 2026. QuantCode-Bench: A Benchmark for Evaluating the Ability of Large Language Models to Generate Executable Algorithmic Trading Strategies. *arXiv preprint arXiv:2604.15151* (2026).
- [8] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, et al. 2023. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. In *International Conference on Machine Learning (ICML)*. arXiv:2211.11501.
- [9] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2305.01210.
- [10] Alejandro Lopez-Lira and Yuehua Tang. 2023. Can ChatGPT Forecast Stock Price Movements? Return Predictability and Large Language Models. *arXiv preprint arXiv:2304.07619* (2023).
- [11] André F. Perold. 1988. The Implementation Shortfall: Paper versus Reality. *The Journal of Portfolio Management* 14, 3 (1988), 4–9. <https://doi.org/10.3905/jpm.1988.409150>
- [12] Yuchen Tian, Weixiang Yan, Qian Yang, Xuandong Zhao, et al. 2024. Code-Halu: Investigating Code Hallucinations in LLMs via Execution-based Verification. *arXiv preprint arXiv:2405.00253* (2024).
- [13] U.S. Securities and Exchange Commission. 2013. In the Matter of Knight Capital Americas LLC, Administrative Proceeding File No. 3-15570, Release No. 34-70694. <https://www.sec.gov/litigation/admin/2013/34-70694.pdf>.
- [14] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, et al. 2023. BloombergGPT: A Large Language Model for Finance. *arXiv preprint arXiv:2303.17564* (2023).
- [15] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. 2023. FinGPT: Open-Source Financial Large Language Models. *arXiv preprint arXiv:2306.06031* (2023).
- [16] Ziyao Zhang, Yanlin Wang, Chong Wang, Jiachi Chen, and Zibin Zheng. 2024. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *arXiv preprint arXiv:2409.20550* (2024).
- [17] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, et al. 2024. Big-CodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv preprint arXiv:2406.15877* (2024).

A $\tau_{\max}$ DISTRIBUTION

[TODO: author list and order must be fixed before the abstract deadline]

MintEval: Do LLMs Implement the Trading Strategy You Asked For?